

Name: Supriya Gaslapati

Reg No: 23MIS7049

S101: B1+TB1

Digital Assignment - 1

Q) case study: Simple login and profile system

A website provides a login system for users.

After login, users can view their profile information stored on the server.

Explain: 1. How PHP sessions and cookies manage user authentication

2. How arrays and JSON can store user data.

3. How PHP filters and validation ensure secure input handling.

1) cookies

→ cookies are small pieces of data stored on the user's browser.

→ when a user logs in, the server can store a cookie (e.g.: user_id or session ID)

→ on future requests, the browser automatically sends the cookie back to the server.

```
ex: setcookie("user", "Supriya", time() + 3600);
```

```
//valid for 1 hour.
```

Sessions

- Sessions stores data on server side
- A unique session ID is created and stored on a cookie (usually PHPSESSID).
- The server uses this ID to identify the user.

login process

1. user enters username and password.
2. server verifies credentials
3. If valid:

```
session_start();
```

```
$_SESSION['user'] = $username;
```

4. on each page:

```
session_start();
```

```
if (isset($_SESSION['user'])) {
```

```
}    echo "Welcome". $_SESSION['user'];
```

22) Arrays in PHP

- used to store details like name, email, age

```
ex: $user = array("name" => "supriya",
```

```
    "email" => "supriya@gmail.com",
```

```
    "age" => 21
```

```
);
```

JSON (JavaScript Object Notation)

- used to store and transfer data between client and server.

- lightweight and easy to read.

Convert array to JSON:

```
$json_data = json_encode($user);
```

convert JSON to Array :

```
$data = json_decode($json_data, true);
```

Use in profile system

- store user data in JSON file or database.
- send profile data to frontend using JSON
- Display user info dynamically.

Q3) why validation is needed

→ prevents attacks like:

SQL injection

XSS (cross-site scripting)

invalid data entry.

PHP filters

PHP provides built-in filters for sanitization and validation

```
$name = filter_input(INPUT_POST, "name",  
    FILTER_SANITIZE_STRING);
```

```
$email = filter_input(INPUT_POST, "email",  
    FILTER_SANITIZE_EMAIL);
```

validation check

```
if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
```

```
    echo "valid email";
```

```
} else {
```

```
    echo "invalid email";
```

```
}
```

Other validation Techniques

1. Empty check

```
if (empty({username})) {  
    echo 'username required';  
}
```

2. Length check

```
if (strlen({password}) < 6) {  
    echo "password too short";  
}
```

3. password security.

```
{hashed-password} = password-hash({password,  
    PASSWORD_DEFAULT});
```

4. prevent XSS

```
echo htmlspecialchars({input});
```